CourseFit — Hints

> Read the story and try to build it first. These hints point you at the pieces and the easy-to-miss parts, without giving the whole answer. The full code is in the complete solution.

Hint 1 — the pieces, and the trick that makes it testable

The same shape as JobMatch, four jobs:

- `src/schemas.py` — `RequiredPrereqs(prereqs: List[str] = [])` and `ReadinessReport(score, met, missing)`.
- `src/matcher.py` — `check_prereqs(required, distance, cutoff=0.8)`. **Build and test this first.** The trick: it takes a **`distance` function**, not a Chroma collection — so you pass a fake `distance(prereq)` in tests and a real one in the pipeline.
- `src/rag/` — `build_background_store(background)` (chunk + embed + Chroma) and `best_distance(collection, prereq)` (Doc08).
- `src/extractor.py` — `extract_required_prereqs(model, course)` with `with_structured_output`.

Build order: schemas → matcher (test it) → RAG → extractor → pipeline.

Hint 2 — the engine and the one lambda

`check_prereqs` is the decision, deterministic:

```
check_prereqs(required, distance, cutoff=0.8):
    met, missing = [], []
    for prereq in required:
        if distance(prereq) <= cutoff: met.append(prereq)
        else: missing.append(prereq)
    total = len(required)
    score = 100 if total == 0 else round(100 * len(met) / total)
    return ReadinessReport(score=score, met=met, missing=missing)
```

(Write the score with an `if`, not a ternary.) Note the empty case returns **100** (no prerequisites = ready), not a divide-by-zero. In the pipeline, bridge the two-argument `best_distance` to the one-argument `distance` the engine wants with a closure:

```
def distance(prereq):
    return best_distance(collection, prereq)
report = check_prereqs(required, distance, cutoff)
```

Test `check_prereqs` with a fake distance (`{"linear algebra": 0.2, "python": 0.1}`): `["Linear algebra","Python","Statistics"]` gives 2 met, 1 missing, score 67; `[]` gives 100. The whole file set is in the complete solution.